

```

#include <stdio.h>
#include <stdlib.h>
#include <termios.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdbool.h>

// Constante
#define TAILLE 12 // 12 par 12
#define TAILLE_DEPLACEMENTS 999

// Types
typedef char t_tableau[TAILLE][TAILLE];
typedef char typeDeplacements[TAILLE_DEPLACEMENTS];
typedef struct{
    char cardep [TAILLE_DEPLACEMENTS];
    int nbDep;
}t_deplacements;

// Variables globales
int pos_x_joueur = 0;
int pos_y_joueur = 0;
int compteur = 0;

// Variables pour condition de victoire
int nbCibles = 0;
int placementCorrect = 0;

// prototypes
char convertir_mv_touche(char cardep);
void charger_partie(t_tableau plateau, char fichier[]);
void afficher_plateau(t_tableau plateau);
void deplacer(t_tableau plateau, t_tableau copie, char touche);
int nombre_cibles(t_tableau plateau);
bool gagne(int nbcible, int placementbon);
void perso_sur_cibles(t_tableau plateau, t_tableau copie);
void chargerDeplacements(typeDeplacements t, char fichier[], int *nb);
t_deplacements copier_tab_dep(char copydep[], int nb);
void enregistrer_dep(t_deplacements dep, char fichier[]);

int main() {

    t_tableau plateau;
    t_tableau copie;
    typeDeplacements t;
    t_deplacements t_copy;

    bool partiegagne = false;
    int nbdeplacementsFic;

```

```

int i = 0;
int j = 0;
int compteurtemp = 0;
char touche;

char fichier[100];
char déplacements[100];

printf("Veuillez saisir le nom d'un fichier : ");
scanf("%20s", fichier);

printf("Veuillez saisir le nom du fichier de déplacements : ");
scanf("%20s", déplacements);

charger_partie(plateau, fichier);

for (int i = 0; i < TAILLE; i++)
    for (int j = 0; j < TAILLE; j++)
        copie[i][j] = plateau[i][j];

chargerDeplacements(t, déplacements, &nbdeplacementsFic);
t_copy = copier_tab_dep(t, nbdeplacementsFic);

nbCibles = nombre_cibles(copie);
placementCorrect = 0;

for (int i = 0; i < TAILLE; i++)
    for (int j = 0; j < TAILLE; j++)
        if (plateau[i][j] == '*')
            placementCorrect++;

afficher_plateau(plateau);

while (!partiegagne && i < nbdeplacementsFic) {

    touche = convertir_mv_touche(t[i]);
    usleep(250000);

    if (touche != '\0') {

        compteurtemp = compteur;
        deplacer(plateau, copie, touche);

        if (compteurtemp == compteur) {
            j = 0;
            while (j < t_copy.nbDep - 1) {
                t_copy.cardep[j] = t_copy.cardep[j + 1];
            }
        }
    }
}

```

```

        j++;
    }
    t_copy.nbDep--;
}
else{
    compteurtemp = compteur;
}

perso_sur_cibles(plateau, copie);
afficher_plateau(plateau);

if (gagne(nbCibles, placementCorrect)) {

    char fichierDepSauvegarde[100] = "deplacements_sauvegarde.dep";
    enregistrer_dep(t_copy, fichierDepSauvegarde);

    printf("La suite de déplacements %s est une solution pour la partie %s.\n\n",
        fichier, déplacements);
    printf("Elle contient %d déplacements\n", nbdeplacementsFic);
    printf("Nombre total de déplacements : %d\n", compteur);
    printf("Position dans le fichier de déplacements : %d\n\n", i + 1);
    printf("Votre fichier de déplacements a été sauvegardé dans : %s\n", fichierDepSauvegarde);

    partiegagne = true;
}
}
i++;
}

if (!partiegagne) {
    printf("La suite de déplacements %s n'est pas \n une solution pour la partie %s.\n",
        fichier, déplacements);
}

return 0;
}

char convertir_mv_touche(char cardep) {

    if (cardep == 'z' || cardep == 'Z') return 'z';
    if (cardep == 's' || cardep == 'S') return 's';
    if (cardep == 'q' || cardep == 'Q') return 'q';
    if (cardep == 'd' || cardep == 'D') return 'd';

    return '\0';
}

```

```

void afficher_plateau(t_tableau plateau) {

    system("clear");

    for (int i = 0; i < TAILLE; i++) {
        for (int j = 0; j < TAILLE; j++)
            printf("%c", plateau[i][j]);
        printf("\n");
    }

    printf("\nDéplacements effectués : %d\n", compteur);
    printf("Cibles totales : %d\n", nbCibles);
    printf("Caisses placées : %d\n", placementCorrect);
}

void charger_partie(t_tableau plateau, char fichier[]) {

    FILE *f;
    char finDeLigne;

    f = fopen(fichier, "r");
    if (f == NULL) {
        printf("ERREUR SUR FICHER");
        exit(EXIT_FAILURE);
    }

    for (int ligne = 0; ligne < TAILLE; ligne++) {
        for (int colonne = 0; colonne < TAILLE; colonne++) {
            fread(&plateau[ligne][colonne], sizeof(char), 1, f);
            if (plateau[ligne][colonne] == '@') {
                pos_x_joueur = colonne;
                pos_y_joueur = ligne;
            }
        }
        fread(&finDeLigne, sizeof(char), 1, f);
    }

    fclose(f);
}

void deplacer(t_tableau plateau, t_tableau copie, char touche) {

    int deltaX = 0, deltaY = 0;
    int nouvelleX, nouvelleY, doubleX, doubleY;
    char caseDevant, caseDerriere;
    bool joueurEtaitSurCible;

```

```

if (touche == 'z' || touche == 'Z') deltaY = -1;
else if (touche == 's' || touche == 'S') deltaY = 1;
else if (touche == 'q' || touche == 'Q') deltaX = -1;
else if (touche == 'd' || touche == 'D') deltaX = 1;
else return;

nouvelleX = pos_x_joueur + deltaX;
nouvelleY = pos_y_joueur + deltaY;
doubleX = pos_x_joueur + 2 * deltaX;
doubleY = pos_y_joueur + 2 * deltaY;

if (nouvelleX < 0 || nouvelleX >= TAILLE || nouvelleY < 0 || nouvelleY >= TAILLE)
    return;

caseDevant = plateau[nouvelleY][nouvelleX];

if (doubleX >= 0 && doubleX < TAILLE && doubleY >= 0 && doubleY < TAILLE)
    caseDerriere = plateau[doubleY][doubleX];
else
    caseDerriere = '#';

joueurEtaitSurCible = (copie[pos_y_joueur][pos_x_joueur] == '.');

if (caseDevant == ' ' || caseDevant == '.') {

    plateau[pos_y_joueur][pos_x_joueur] = joueurEtaitSurCible ? '.' : ' ';
    plateau[nouvelleY][nouvelleX] = (copie[nouvelleY][nouvelleX] == '.') ? '+' : '@';
    pos_x_joueur = nouvelleX;
    pos_y_joueur = nouvelleY;
    compteur++;
}

else if ((caseDevant == '$' || caseDevant == '*') &&
        (caseDerriere == ' ' || caseDerriere == '.')) {

    if (caseDevant == '*') placementCorrect--;

    plateau[pos_y_joueur][pos_x_joueur] = joueurEtaitSurCible ? '.' : ' ';
    plateau[nouvelleY][nouvelleX] = (copie[nouvelleY][nouvelleX] == '.') ? '+' : '@';

    if (copie[doubleY][doubleX] == '.') {
        plateau[doubleY][doubleX] = '*';
        placementCorrect++;
    } else {
        plateau[doubleY][doubleX] = '$';
    }
}

pos_x_joueur = nouvelleX;

```

```

        pos_y_joueur = nouvelleY;
        compteur++;
    }
}

```

```

int nombre_cibles(t_tableau plateau) {

```

```

    int nb = 0;

```

```

    for (int i = 0; i < TAILLE; i++)
        for (int j = 0; j < TAILLE; j++)
            if (plateau[i][j] == '.')
                nb++;

```

```

    return nb;
}

```

```

bool gagne(int nbcible, int placementbon) {
    return nbcible == placementbon;
}

```

```

void perso_sur_cibles(t_tableau plateau, t_tableau copie) {

```

```

    for (int i = 0; i < TAILLE; i++)
        for (int j = 0; j < TAILLE; j++)
            if (copie[i][j] == '.' && plateau[i][j] == ' ')
                plateau[i][j] = '.';
}

```

```

void chargerDeplacements(typeDeplacements t, char fichier[], int *nb) {

```

```

    FILE *f;
    char dep;

```

```

    *nb = 0;
    f = fopen(fichier, "r");

```

```

    if (f == NULL) {
        printf("FICHIER NON TROUVE\n");
        return;
    }

```

```

    while (fread(&dep, sizeof(char), 1, f)) {
        t[*nb] = dep;
        (*nb)++;
    }

```

```

    fclose(f);

```

```
}
```

```
t_deplacements copier_tab_dep(char copydep[], int nb) {
```

```
    t_deplacements déplacements;
```

```
    déplacements.nbDep = 0;
```

```
    for (int i = 0; i < nb; i++) {
```

```
        déplacements.cardep[i] = copydep[i];
```

```
        déplacements.nbDep++;
```

```
    }
```

```
    return déplacements;
```

```
}
```

```
void enregistrer_dep(t_deplacements dep, char fichier[]) {
```

```
    FILE *f = fopen(fichier, "w");
```

```
    if (!f) {
```

```
        printf("Erreur lors de la sauvegarde des déplacements.\n");
```

```
        return;
```

```
    }
```

```
    for (int i = 0; i < dep.nbDep; i++)
```

```
        fputc(dep.cardep[i], f);
```

```
    fclose(f);
```

```
}
```